GitHub Parent–Child Terraform Automation — Enterprise Platform Blueprint

---

Executive summary

This blueprint prescribes a secure, enterprise-ready GitHub parent–child repository architecture to automate AWS infrastructure provisioning using Terraform. It focuses on:

- Clear parent (platform) vs child (service) repo responsibilities
- Reusable GitHub Actions workflows (workflow_call) to standardize CI/CD
- OIDC-based AWS authentication (no long-lived secrets)
- Remote state and artifact architecture (S3 + DynamoDB + KMS)
- Policy-as-code, branch protection, and auditability
- Multi-account/multi-environment scalability with dependency-aware orchestration
- Robust rollback and disaster recovery processes
- KPIs and maturity metrics to measure platform effectiveness

This blueprint is prescriptive and executable: it contains repository layouts, concrete workflow templates, IAM trust patterns, remote-state layouts, orchestration models, governance controls, and implementation checklists.

Target-state reference architecture

- AWS multi-account landing zone (recommended accounts):
  - Management / Control Plane
  - Security / Logging
  - Shared Services (CI/CD artifacts, central S3 state bucket(s), DynamoDB locking, KMS)
  - Sandbox / Dev
  - Staging
  - Production
- GitHub organization structure:
  - org/platform (Parent repo)
    - Central reusable-workflows (/ .github/workflows/*)
    - Module registry index / documentation / service catalog
    - Orchestration workflows / promotion tools / dashboards
  - org/- (Child repos)
    - Service-specific Terraform code, examples, tests
    - Call into platform reusable workflows for plan/apply flows

- Runtime flow summary:
  - Child repo PR triggers platform `terraform-plan` reusable workflow (child initiates).
  - Plan output (plan.json + metadata) is uploaded to a secure central S3 artifact store (short-lived creds via OIDC).
  - Environments use GitHub Environment protection for apply gating.
  - Parent orchestrator aggregates plans for cross-repo promotions; approvals initiate applies in dependency order via workflow_call or repository_dispatch.
  - Apply jobs assume narrow IAM roles (per-account/per-environment/per-repo) via OIDC.
- Key security controls:
  - GitHub → AWS OIDC trust configured per-account with repo/ref restrictions
  - IAM roles scoped to required resource ARNs and actions (least privilege)
  - S3 bucket(s) with SSE-KMS, bucket policy, public access blocked, logging, versioning
  - DynamoDB locking table in shared account for Terraform state locks
  - Policy as code enforced in plan stage (Conftest/OPA, tfsec, checkov)
  - CloudTrail + GitHub audit logs + resource tags linking run metadata

---

Parent repository orchestration design (org/platform)

Responsibilities:

- Host versioned, audited reusable workflows for plan, policy, apply, promote, and audit.
- Operate a manifest and dependency graph for multi-repo orchestration.
- Provide platform documentation, onboarding, templates, and module registry pointers.
- Maintain CI tests for reusable workflows (canary releases).
- Manage central artifact & state infrastructure (S3 buckets, DynamoDB, KMS config).
- Provide operator-level orchestration workflows for bulk promotions.

Key components:

- / .github/workflows/
  - terraform-plan.yml (reusable)
  - terraform-apply.yml (reusable)
  - terraform-policy.yml (reusable)
  - terraform-promote.yml (orchestrator)
- /docs/ — onboarding, runbooks, module catalog
- /manifests/ — YAML manifest listing child repos and dependency metadata
- /scripts/orchestrator/ — automation to read manifests, collect plans, and trigger applies
- /tests/ — unit and integration tests for workflows and modules

Design decisions:

- Version reusable workflows with tags (v1, v1.1, v2) to avoid breaking child repos.
- CI for platform repo must include tests that run child-repo-style calls against canary child repos.

- Parent orchestrator must never assume an overly-permissive role that bypasses child repo ownership — parent triggers child apply workflows that run within child OIDC trust context.

Example parent repo layout:

- platform/
  - .github/workflows/terraform-plan.yml
  - .github/workflows/terraform-apply.yml
  - manifests/services.yaml
  - docs/
  - scripts/orchestrator.py
  - tests/

Child repository standards

Purpose:

- One logical service/component per repo (e.g., infra/vpc, infra/eks, infra/rds).
- Minimal orchestration inside child repo; use parent reusable workflows for CI/CD standardization.

Mandated files/structure:

- / main.tf, variables.tf, outputs.tf, versions.tf
- / modules/ (only if local submodules exist; prefer registry modules)
- / examples/ (recommended)
- / .github/workflows/ (child-specific entrypoint that calls parent reusable workflow via uses:)
- / CODEOWNERS (for resource-specific approvals)
- / README.md (module usage, environment mapping, owner contact)
- / .github/terraform.tfvars.example (to show required env vars or variables)
- / tests/ (unit tests, plan tests)

Child repo behavior:

- On PR:
  - Call platform's terraform-plan reusable workflow (workflow_call).
  - Present plan summary as PR comment with structured metadata.
  - Provide required status checks for branch protection.
- On merge:
  - Upload plan artifact (secure S3) and mark plan as “ready for apply”.
  - Apply must only run via environment-protected workflow (manual approval) or upon parent orchestrator triggering.

Standards:

- Pin module versions (registry or git tag). No floating references to trunk.
- Tag commit releases when moving to production.
- Use environment-scoped secrets only when necessary.

- Ensure Terraform backend config is not hard-coded credentials (use CI-injected backend configs).

Ownership model:

- Product teams own child repos (code & runbook).
- Platform team owns reusable workflows, module registry, platform infra.

GitHub Actions reusable workflow templates

Below are concise, production-focused templates (abbreviated for readability). Use these as canonical starting points and version them in the platform repo.

1. terraform-plan.yml (reusable, called by child repos via workflow_call)

- Inputs:
  - workspace: (string) environment name (dev/stg/prod)
  - aws_account_id: string
  - role_to_assume: string (arn)
  - tf_working_dir: string (path in repo)
  - s3_plan_bucket: string (central S3 bucket)
  - plan_key_prefix: string (prefix for plan artifacts)
  - tf_backend_config: optional map or object (backend bucket/key info)
- Outputs:
  - plan_s3_key: path/key for stored plan JSON
  - plan_sha256: checksum of plan file
  - plan_summary: short summary text

Key steps (conceptual):

- Checkout code
- Setup Terraform CLI
- Run fmt, validate, tflint, tfsec/checkov, conftest
- Run terraform init with backend config from inputs
- terraform plan -out=tfplan
- terraform show -json tfplan > plan.json
- Compute checksum of plan.json
- Assume role via aws-actions/configure-aws-credentials (OIDC) to write plan.json into central S3 (encrypted with KMS)
- Post PR comment with summary and attach plan metadata
- Output plan_s3_key and plan_sha256

Sample YAML (abridged):

- Note: In production, include full error handling, retry, secrets masking, and modularized action steps. This is a canonical template — store in platform repo and tag.

```
# platform/.github/workflows/terraform-plan.yml (reusable)
on:
```

```
  workflow_call:
    inputs:
      workspace:
        required: true
        type: string
      aws_account_id:
        required: true
        type: string
      role_to_assume:
        required: true
        type: string
      tf_working_dir:
        required: true
        type: string
      s3_plan_bucket:
        required: true
        type: string
      plan_key_prefix:
        required: true
        type: string

jobs:
  plan:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v2
        with:
          terraform_version: 1.5.0

      - name: Terraform fmt
        run: terraform -chdir=${{ inputs.tf_working_dir }} fmt -check

      - name: Terraform validate
        run: terraform -chdir=${{ inputs.tf_working_dir }} init -input=

      - name: Static security checks
        run: |
          # tflint, tfsec, checkov, conftest
          tflint --init
          tflint ${ { inputs.tf_working_dir } } || true
          tfsec ${ { inputs.tf_working_dir } } || true

      - name: Terraform plan
        env:
          TF_IN_AUTOMATION: "true"
        run: |
          terraform -chdir=${{ inputs.tf_working_dir }} init -input=fal
            -backend-config="bucket=${{ inputs.tf_backend_bucket || ''
            # Backend config should be injected dynamically by CI as en
```

```
        terraform -chdir=${{ inputs.tf_working_dir }} plan -input=fal
        terraform -chdir=${{ inputs.tf_working_dir }} show -json tfpl

    - name: Compute plan checksum
      id: checksum
      run: |
        sha256sum plan.json | awk '{print $1}' > plan.sha256
        echo "plan_sha256=$(cat plan.sha256)" >> $GITHUB_OUTPUT

    - name: Assume role to upload plan to S3 (OIDC)
      uses: aws-actions/configure-aws-credentials@v3
      with:
        role-to-assume: ${{ inputs.role_to_assume }}
        role-duration-seconds: 900
        aws-region: us-east-1

    - name: Upload plan.json to S3
      run: |
        PLAN_KEY="${{ inputs.plan_key_prefix }}/$(echo $GITHUB_REPOSI
        aws s3 cp plan.json s3://${{ inputs.s3_plan_bucket }}/${PLAN_
        echo "plan_s3_key=${PLAN_KEY}" >> $GITHUB_OUTPUT

    - name: Post PR comment (optional)
      # Use GitHub API to post plan summary to originating PR (if exi
      run: |
        # use gh CLI or GitHub REST API to post plan summary
```

1. terraform-apply.yml (reusable, environment-protected; expects pre-created plan in S3)

- Inputs:
  - plan_s3_key
  - plan_sha256
  - workspace
  - role_to_assume
  - tf_working_dir
  - s3_plan_bucket
- Behavior:
  - Requires environment protection in calling repo (target environment: production or staging).
  - Uses OIDC to assume narrow role to perform apply in target AWS account.
  - Downloads plan.json from S3, verifies checksum, converts to terraform binary plan via terraform show -json -> tfplan (or use plan binary stored earlier), performs terraform apply -auto-approve .
  - Records apply metadata: GitHub run id, approver id, commit sha in AWS session name & resource tags.

Example (abridged):

```
# platform/.github/workflows/terraform-apply.yml (reusable)
on:
  workflow_call:
```

```
    inputs:
      plan_s3_key:
        required: true
      plan_sha256:
        required: true
      tf_working_dir:
        required: true
      role_to_assume:
        required: true
      s3_plan_bucket:
        required: true

jobs:
  apply:
    runs-on: ubuntu-latest
    environment:
      name: ${{ inputs.workspace }}  # must map to GitHub Environment p
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Assume role for apply (OIDC)
        uses: aws-actions/configure-aws-credentials@v3
        with:
          role-to-assume: ${{ inputs.role_to_assume }}
          aws-region: us-east-1
          role-duration-seconds: 900

      - name: Download plan artifact from S3
        run: |
          aws s3 cp s3://${{ inputs.s3_plan_bucket }}/${{ inputs.plan_s
          echo "Downloaded plan file"

      - name: Verify plan checksum
        run: |
          echo "${{ inputs.plan_sha256 }}  -" > expected.sha256
          sha256sum -c expected.sha256 < plan.json || (echo "Checksum m

      - name: Recreate tfplan (if stored as json->binary) or apply dire
        run: |
          # If stored as binary tfplan, simply apply
          # If stored as JSON, reconstruct or run apply via terraform a
          terraform -chdir=${{ inputs.tf_working_dir }} init -input=fal
          terraform -chdir=${{ inputs.tf_working_dir }} apply -input=fa

      - name: Tag apply in logs / session
        run: |
          # Set tags on resources if required via provider or append me
```

Operational guidance:

- Do not grant apply workflow more privilege than necessary in the IAM role.
- Require GitHub Environment approval for production applies.

- Use job-level concurrency and cancellation to prevent multiple simultaneous applies for same state.

Versioning:

- Tag reusable workflows in platform repo (e.g., uses: org/platform/.github/workflows/terraform-plan.yml@v1).
- Release patch versions for non-breaking changes; increment minor/major for contract changes.

OIDC-based AWS authentication model

Goals:

- Eliminate long-lived AWS keys.
- Use short-lived credentials via GitHub OIDC tokens (AssumeRoleWithWebIdentity).
- Restrict trust to specific repos/branches/workflows.

AWS setup per account:

1. Create IAM OIDC provider for token.actions.githubusercontent.com (one per AWS account).
2. Create IAM role for CI operations per environment per repo or per team:
    - Naming convention: arn:aws:iam:::role/terraform-{env}-{repo|-team}
    - Trust policy restricts allowed subjects (sub claim) and audience.

Sample trust policy (restrict by repo and branch):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::ACCOUNT_ID:oidc-provider/token.actio
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com
          "token.actions.githubusercontent.com:sub": "repo:my-org/my-re
        }
      }
    }
  ]
}
```

For org-level patterns where multiple repos should assume a role, use more flexible conditions, but combine with workflow or repo_name claims to reduce risk:

- Example allow multiple repos:

    - "StringLike": { "token.actions.githubusercontent.com:sub": ["repo:my-org/*:ref:refs/heads/main", "repo:my-org/*:ref:refs/tags/*"] }

- Recommend adding additional claims / constraints in trust policy (e.g., require job/actor to be in list).

Role session naming:

- Set session name to include: github-actions/{repo}/{run_id}/{actor} so CloudTrail records can tie actions back to a GitHub run & actor.

IAM policy design:

- Create tightly-scoped policies per role that only allow actions on resource ARNs belonging to that module/service.
- Use condition keys that check for resource tags where possible.

OIDC best practices:

- Do not allow forks to assume roles (restrict by ref and repo).
- Limit audience to "sts.amazonaws.com".
- Rotate trust policies as platform changes; log trust relationships.

Remote state architecture

Principles:

- One state file per service per environment (e.g., infra/vpc-prod).
- Use S3 with SSE-KMS and versioning for Terraform state storage.
- Use DynamoDB for lock management (single DynamoDB per region/account for simplicity).
- Keep state buckets in Shared Services account, or use per-account buckets with cross-account roles — choose based on security posture.

S3 bucket naming & layout:

- Bucket name: org-terraform-state-{region}
- Object keys: ///.tfstate
- Example: 123456789012/prod/vpc/vpc.tfstate

S3 bucket configuration:

- Server-side encryption with KMS (SSE-KMS)
- Block public access
- Access logging enabled
- Versioning enabled (for state recovery)
- Bucket policy restricting access to specific IAM roles (platform & service roles)

DynamoDB locking table:

- Table name: terraform-state-locking
- Partition key: LockID
- Global table per region with restricted access by IAM roles

Backend config in Terraform:

- Do not hardcode credentials in backend. Inject backend config via CI env vars or via a small template file generated during the plan/apply workflow, which is not checked in.

Backend example block (terraform code):

```
terraform {
  backend "s3" {
    bucket         = "org-terraform-state-us-east-1"
    key            = "123456789012/prod/vpc/vpc.tfstate"
    region         = "us-east-1"
    kms_key_id     = "arn:aws:kms:us-east-1:123456789012:key/abc"
    dynamodb_table = "terraform-state-locking"
    encrypt        = true
  }
}
```

Artifact (plan) storage:

- Use a central S3 bucket for plan.json artifacts:
  - Plan key pattern: plans///-<run_id>.plan.json
  - Apply uses the same plan key to guarantee apply equality
  - Plans must be stored encrypted & versioned; restricted read/write via IAM roles only

Cross-account considerations:

- If centralizing state in shared account, setup IAM roles with narrowly scoped trust for per-account operations.
- Alternatively, keep per-account S3 buckets with similar configuration — whichever aligns with org security policies.

Governance and compliance framework

Controls:

- Branch protection:
  - Protect main/prod branches; require PR reviews, required status checks (plan, policy checks), disallow force pushes.
- CODEOWNERS:
  - Define code owners for sensitive paths (IAM, networking).
  - Require those teams' reviews for changes to those paths.
- GitHub Environments:
  - Setup environment protection for dev/staging/production with required reviewers and wait timers.
- Policy-as-code:

    - Enforce policies in plan stage:
        - tfsec/checkov for security scanning
        - conftest (OPA/Rego) for org-specific rules (tags, encryption, AMI lists)
        - Sentinel if using Terraform Cloud/Enterprise
- Audit trail:
    - Ensure CloudTrail and AWS Config are enabled in all accounts.
    - Use GitHub audit logs and link run id -> AWS session name -> CloudTrail.
    - Tag resources with metadata: org, repo, commit, run_id, approver.
- Approvals:
    - Use GitHub required reviewers and environment-level approvers for production.
    - Record approver identity as part of the apply job metadata and AWS session name.

Policy-as-code sample checks:

- No public S3 buckets
- Encryption required for all storage
- IAM policies must not use "*" on both Action and Resource
- Enforce mandatory tags: owner, cost_center, environment, project

Compliance reporting:

- Collect CI results into a central dashboard (S3 + Lambda + Kibana/Elasticsearch or a SaaS) that shows policy violations, unresolved plans, and pending promotions.
- Regular audits to ensure state buckets, KMS keys, and roles follow least privilege.

CI/CD sequencing strategy

Core pattern:

- Plan → Policy Check → Store Plan → Request Approval → Apply

Job sequencing details:

- Pre-checks (fmt, validate, lint, security scans) run before plan.
- Plan job outputs: structured JSON + checksum + upload to S3.
- Promotion (apply) gated by:
    - GitHub Environment manual approval
    - Or parent orchestrator aggregated approval for multi-repo promotions

Parallelization & matrix:

- Use strategy.matrix for region or environment parallel planning.
- Sequence applies using needs or parent orchestrator DAG following dependency graph:
    - Example: jobs: plan-vpc needs: [] (parallel)
    - apply-subnet needs: plan-vpc

Artifact sharing:

- Use S3 for cross-repo artifact sharing (plans)
- Use ephemeral, short-lived presigned URLs for limited access, but only when ephemeral token usage is validated
- Avoid cross-repo GitHub upload-artifact usage (repo-scoped)

Promotion orchestration:

- Child-initiated apply model:
  - Child prepares plan and uploads to S3
  - Parent orchestrator lists pending plans, aggregator produces release candidate
  - Approver approves release and parent triggers child apply workflows in correct order
- Parent-initiated model:
  - Parent triggers child plan runs via repository_dispatch, collects plans, then applies in order

Sequencing patterns:

- Use manifest with dependency graph in parent:
  - Topologically sort graph to determine apply order
  - Example: vpc -> subnet -> bastion -> ecs
- Ensure per-state locking to avoid concurrency.

Failure handling:

- Apply failures should not automatically roll back other modules — isolate failures and retry after remediation.
- Parent orchestrator must have configurable retry/backoff and manual pause points for operator intervention.

Rollback and disaster recovery model

Key principles:

- Always keep state backups and object versioning enabled.
- Maintain release tags and module version pinning to enable version-based rollbacks.
- Have well-documented runbooks for emergency state restore.

Recovery mechanisms:

1. Restore Terraform state:
   - Restore S3 object to previous version (using S3 versioning) and re-run terraform apply with corresponding config or module versions.
   - DynamoDB locking must be cleared (carefully) if failed locks prevent operations.
2. Roll back module changes:

    - Revert child repo to previous module versions and run plan/apply in non-prod then prod (after approval).

3. Resource-level rollback:

    - For destructive changes (resource deletes), use recovered state to re-create resources or restore from backups (RDS snapshots, S3 backups).

4. Orchestrated bulk rollback:

    - Use parent orchestrator to update manifests to previous commit/module versions and perform coordinated applies across child repos.

Runbooks (high-level):

- Incident detection:
    - Detect via scheduled drift checks or alerts from failed applies or CloudWatch alarms.
- Containment:
    - Pause parent orchestrator operations, prevent further applies via GitHub environment lock.
- Recovery:
    - Restore state from S3 version; reapply or re-import resources as necessary.
    - If needed, re-run terraform apply with roll-back commit or module version.
- Postmortem:
    - Create an incident postmortem and update policies to prevent recurrence.

Automated backups & tests:

- Periodic automated backups of state to a different bucket/account (disaster isolation)
- Periodic restore tests (canary restore) to validate backup integrity and runbook accuracy

Emergency access:

- Establish emergency operator roles with strict controls and logged sessions (MFA enforceable via AWS IAM).
- Emergency procedures require separate approvals & audit logging.

Platform scalability approach

Scalability dimensions:

- Account scale (dozens to hundreds of AWS accounts)
- Repo count (hundreds of child repos)
- Workflow run scale (concurrent plans/applies)

Patterns & recommendations:

- Multi-repo (parent-child) model scales better for autonomous teams and multi-account AWS; recommended for enterprise.
- Use manifest-driven orchestration to coordinate thousands of child repos via batching and dependency grouping.
- Use ephemeral self-hosted runners in AWS (ECS/EKS/Auto-scaling EC2/Spot) to scale long-running or heavy jobs:
    - Self-hosted runners should run in Shared Services account with IAM instance role restricted to runner needs.
    - Use autoscaling groups for on-demand capacity; separate runner pools for heavy Terraform jobs vs light CI tasks.

Platform throughput techniques:

- Plan jobs can be highly parallelized. Use matrix and distributed runs.
- Apply jobs require coordination; use worker pools or orchestrator to schedule sequentially for dependent sets.
- Caching: cache providers/plugins between runs for speed where feasible.

Observability & telemetry:

- Central logging of workflow runs to a telemetry system (CloudWatch, ELK, Datadog).
- Central dashboard that shows plan status, pending approvals, and apply health.

Operational scalability:

- Canary rollout of reusable workflows: first update small set of child repos via vX-canary tags before broad rollout.
- Enforce workflow reuse with policy checks (prevent child repos from running apply without platform workflow).

Cost controls:

- Use short-lived runners, spot capacity for non-critical runs.
- Limit concurrency usage via GitHub concurrency settings per repo or organization.

KPIs and platform maturity metrics

Operational KPIs:

- Deployment Frequency: number of successful infrastructure applies per time period
- Mean Time to Recovery (MTTR): average time to restore infra after failure
- Mean Time To Approve (MTTA): average time between plan-ready and approval
- Mean Time to Detect (MTTD): average time to detect drift or failing apply
- Change Failure Rate: percent of applies resulting in rollback or manual remediation
- Policy Compliance Rate: percent of plans that pass policy checks pre-apply
- Percentage of repos using pinned module versions
- Percentage of apply operations using OIDC-assumed roles (no long-lived creds)
- Number of state conflicts per period (locking-related incidents)
- Plan-to-apply equality score (percentage where apply used exact plan uploaded)

Platform maturity metrics (levels 0-4):

- Level 0 (Ad-hoc):
  - Manual scripts, long-lived keys, monolithic state
- Level 1 (Standardized):
  - Reusable workflows exist; OIDC on-boarding partial; state per env minimal
- Level 2 (Automated):
  - Parent platform repo with versioned workflows; child repos consistently call workflows; policy-as-code enforced
- Level 3 (Orchestrated):
  - Parent orchestration for cross-repo promotions; dependency manifest; centralized dashboards; strict OIDC use
- Level 4 (Optimized / Self-service):
  - Self-service portal, canary rollout, automated drift remediation, full observability & SLOs, advanced RBAC & fine-grained IAM policies

Targets (example):

- Reach Level 2 within 3 months after starting platform repo
- 90% child repos using pinned modules and platform reusable workflows in 6 months
- Policy compliance rate >= 98%
- Deployment frequency target consistent with org needs (e.g., daily safely for non-prod, weekly for prod)
- Mean Time to Approve < 2 hours for routine changes (org-specific)

CI/CD sequencing — concrete strategies and examples

Use case: multi-repo release promotion across dev -> stg -> prod with dependency graph.

1. Child-initiated plan:

- Each child repo executes plan workflow (platform reusable) and uploads plan artifact to S3 in path: plans///-.plan.json

1. Parent collection & review:

- Parent orchestrator reads manifests and scans S3 for plans matching release-id.
- It assembles a cross-repo change digest and presents a consolidated review UI / PR with links to per-repo plan summaries.

1. Promotion:

- Approver approves consolidated promotion.

- Parent orchestrator triggers apply for child repos in topologically sorted order via:
  - Repository_dispatch event to child apply workflow, or
  - Calling child repo's reusable apply workflow via workflow_call (if permitted).
- Apply runs under child repo's OIDC-assumed role (ensures least privilege). Parent orchestrator should not apply directly to child's state.

Sequential vs parallel:

- For independent sets: apply in parallel (batched) for speed.
- For dependency chains: apply sequentially.

Concurrency & locking:

- Use Terraform state locking (DynamoDB) and per-state concurrency limits in GitHub Actions (concurrency key naming by service + environment).

Sample manifest schema (manifests/services.yaml):

```
services:
  - name: vpc
    repo: org/infra-vpc
    env:
      - dev
      - prod
    depends_on: []
  - name: subnet
    repo: org/infra-subnet
    depends_on:
      - vpc
  - name: eks
    repo: org/infra-eks
    depends_on:
      - vpc
      - subnet
```

Orchestration script (pseudocode):

- Read manifest, build graph, topologically sort, for each node:
  - Trigger apply for node repo for intended env
  - Wait for success or handle failure according to policy (retry, break, escalate)

Rollback and disaster recovery (detailed steps & examples)

1. Immediate response to catastrophic apply failure:

- Pause all related orchestration runs (parent pause feature).
- Notify SRE/Platform and impacted owner(s).
- Inspect GitHub run logs and CloudTrail with session details (use session name to find events).

- If needed, revert Git commit in child repo and trigger apply to re-instate previous known-good state.

1. State restore from S3:

- Identify desired previous tfstate version (S3 object version).
- Copy/restore that object to current tfstate key (or restore to a new key and point backend temporarily).
- Re-run terraform plan to confirm the restored state matches desired infra; apply as required.

1. Data recovery:

- For RDS/S3: use backups/snapshots
- Ensure backup/restore tests are part of DR drills.

1. Testable DR runbook:

- Validate ability to restore state from snapshot in a non-prod recovery account
- Periodic table-top and runbook execution with time-bound goals

1. Rollback via module version:

- Re-pin child repo module versions to previous stable tag, run plan/apply in dev -> stg -> prod following normal promotion.

1. Post-incident:

- Update manifest and platform to block patterns that caused failure.
- Add additional policy-as-code checks if the failure was due to policy drift.

Implementation checklist — step-by-step

Phase 0: Platform foundations

- Create org/platform repo; implement reusable workflows and test harnesses.
- Establish Shared Services AWS account with:
  - S3 state & plan buckets: SSE-KMS, versioning, access logging
  - DynamoDB locking table
  - Central KMS keys (with narrow key policies)
  - CloudTrail & AWS Config

Phase 1: Identity & security

- Configure GitHub OIDC provider in each AWS account.
- Create IAM roles per environment/repo following naming conventions; apply narrow policies.
- Set up trust policies with repo/ref restrictions.

Phase 2: CI/CD rollout (canary)

- Set up sample child repos and adopt platform reusable workflows.
- Version platform workflows and release v1 (canary) for small set of repos.

- Iterate and fix issues.

Phase 3: Governance & policy enforcement

- Configure branch protection and CODEOWNERS in child repos.
- Add policy-as-code tools into plan workflow.
- Configure GitHub Environments for environment-level approvals.

Phase 4: Orchestration & scaling

- Implement parent manifest and orchestration scripts.
- Add dashboards and telemetry.
- Scale runners with autoscaling self-hosted runners or keep GitHub-hosted for simplicity.

Phase 5: DR & maturity

- Implement state backup and restore testing cadence.
- Track KPIs and measure maturity; iterate to Level 3/4.

Practical examples & snippets

1. Plan S3 key pattern:

- plans///-.plan.json
  - Example: plans/release-2026-02/org/infra-vpc/abcd1234-456.plan.json

1. Terraform backend state key pattern:

- states///.tfstate
  - Example: states/123456789012/prod/vpc.tfstate

1. GitHub workflow call (child repo):

```
# child/.github/workflows/ci.yml
name: Terraform CI
on:
  pull_request:
    types: [opened, synchronize, reopened]
  push:
    branches: [main]

jobs:
  plan:
    uses: org/platform/.github/workflows/terraform-plan.yml@v1
    with:
      workspace: dev
      aws_account_id: ${{ secrets.AWS_ACCOUNT_ID }}
      role_to_assume: arn:aws:iam::123456789012:role/terraform-dev-infr
      tf_working_dir: "./"
      s3_plan_bucket: "org-terraform-plans-us-east-1"
      plan_key_prefix: "plans/release-2026-02"
```

1. Sample IAM role policy (least privilege snippet for VPC actions — example only):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowVPCManagement",
      "Effect": "Allow",
      "Action": [
        "ec2:CreateVpc",
        "ec2:DeleteVpc",
        "ec2:CreateSubnet",
        "ec2:DeleteSubnet",
        "ec2:CreateRouteTable",
        "ec2:ModifyVpcAttribute",
        "ec2:Describe*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "AllowS3PutPlan",
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": ["arn:aws:s3:::org-terraform-plans-us-east-1/plans/*"
    }
  ]
}
```

Note: refine to resource ARNs where possible (e.g., use conditions for VPC tags or restrict to particular resource prefixes via tags).

Risks, anti-patterns & mitigations (concise)

- Long-lived AWS credentials in GitHub secrets
  - Mitigation: OIDC-based assume-role
- Centralized "apply" service with excessive rights
  - Mitigation: roles per repo/environment; parent triggers child applies, but child runs in its own OIDC scope
- Single monolithic Terraform state
  - Mitigation: one-state-per-service-per-env; S3 versioning; DynamoDB locking
- Unversioned reusable workflows
  - Mitigation: tag & semver; canary testing & CI for platform repo
- Cross-repo artifact reliance using upload-artifact
  - Mitigation: use S3 central artifact store

- Allowing forks to assume roles
  - Mitigation: trust policy restrictions by sub claim and ref
- Relying on Terraform workspaces for environment boundary
  - Mitigation: separate backend configurations/per-state files

---

Appendices & recommended next steps

Recommended toolset:

- hashicorp/setup-terraform, terraform CLI
- aws-actions/configure-aws-credentials (OIDC)
- tflint, tfsec, checkov, conftest (OPA/Rego)
- Terragrunt (optional orchestration)
- Monitoring: CloudWatch/Datadog/ELK for pipeline & infra telemetry

Suggested immediate next actions:

1. Create platform repo and implement terraform-plan.yml & terraform-apply.yml as versioned artifacts.
2. Configure OIDC providers in two non-prod AWS accounts and set up example IAM roles with restrictive trust policies.
3. Provision shared S3/DynamoDB/KMS for state & plans (dev/test).
4. Convert one child repo to call platform plan workflow and validate full end-to-end (plan -> upload -> apply with environment gating).
5. Implement parent orchestrator manifest and test cross-repo promotion on a small set of services.
6. Roll out to more repos while tracking KPIs.

If you'd like, I can now:

- Provide complete, ready-to-use GitHub Actions YAML templates (full fleshed-out with secrets handling & error trapping).
- Draft concrete IAM trust policy snippets tailored to your org and branch naming rules.
- Produce a sample parent orchestrator script (Python) that reads manifest YAML and issues repository_dispatch events, handles plan collection, and orchestrates apply order.

Which of those would you like me to generate next?